

SIMATS ENGINEERING ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 5

Duration: 1 Hour
Total Marks:50

Annexure A

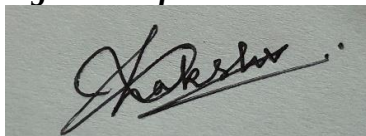
Scenario Based Assignment

Code of Conduct:

I S.LAKSHITA(192524108) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty
- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic
- iii. Analyze the impact of heuristic accuracy on both algorithms
- iv. Discuss how dynamic changes (blocked paths) affect search performance
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. Scenario: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

- i. Explain how Hill Climbing would work in this scenario and identify its limitations
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations
- iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. Scenario: An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. Explain why this problem requires an online search agent instead of offline search

- ii. Analyze the challenges of partial observability and dynamic environments
- iii. Describe how the rover builds and updates its internal model
- iv. Compare online search with uninformed and informed search strategies
- v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

- i. Clearly define variables, domains, and constraints with explanation
- ii. Explain how backtracking search works in this scenario
- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
- iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- i. Explain how the Minimax algorithm models this problem
- ii. Analyze the computational challenges of large game trees
- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
- iv. Design an evaluation function and justify the factors considered
- v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

SIMATS ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CO2 – ASSESSMENT TOOL 2
Scenario Based Assignment

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA17
QUESTIONS: 5 **Duration:** 1 Hour **Total Marks:** 50

Criteria	Weightage	Q1 (10)	Q2 (10)	Q3 (10)	Total Marks (30)
Understanding of Problem Context	20% (2)	2/2	2/2	2/2	6/6
Application of Concepts	30% (3)	3/3	2.5/3	3/3	8.5/9
Problem-Solving Approach	20% (2)	2/2	2/2	2/2	6/6
Accuracy of Solution	20% (2)	2/2	2/2	1.5/2	5.5/6
Clarity	10% (1)	1/1	1/1	1/1	3/3
Total per Question	10 Marks	10/10	9.5/10	9/10	28.5/30 (95/100)

95

100

ANSWERS

1. AI-Powered Drone for Emergency Medicine Delivery

Introduction

An AI-powered drone delivers emergency medicines in a flood-affected area where roads may be blocked, weather changes frequently, and travel costs vary. It must choose safe and efficient routes using informed search algorithms.

i. Greedy Best-First Search (GBFS)

Greedy Best-First Search selects the path that appears closest to the destination using only the heuristic (straight-line distance).

Strengths

- Makes decisions quickly.
- Explores fewer nodes.
- Suitable for urgent situations.

Weaknesses

- Ignores actual travel cost.
- May choose blocked or unsafe routes.
- Does not always find the best path.

Analysis: GBFS is fast but less reliable in uncertain environments.

ii. A* Search

A* uses the function $f(n) = g(n) + h(n)$, where $g(n)$ is the actual cost and $h(n)$ is the estimated cost to the goal.

Advantages

- Considers both cost and distance.
- Finds safer and more efficient routes.
- Produces an optimal solution when the heuristic is accurate.

Limitation

- Requires more computation than GBFS.

Analysis: A* balances speed and path quality, making it suitable for emergency delivery.

iii. Impact of Heuristic Accuracy

- **GBFS:** Highly dependent on heuristic accuracy. Poor estimates may lead to wrong paths.
- **A*:** Accurate heuristics improve speed, while actual cost helps maintain better decisions.

Analysis: A* is less affected by heuristic errors than GBFS.

iv. Effect of Dynamic Changes

- **GBFS:** May need to restart if the chosen path becomes blocked.
- **A*:** Can recalculate routes using updated costs and find better alternatives.

Analysis: A* handles changing environments more effectively.

v. Recommended Algorithm

Recommended: A Search*

Justification

- Finds optimal and safe routes.
- Considers changing costs and obstacles.
- Adapts better to dynamic environments.

- Provides reliable decisions during emergencies.

Conclusion

Both algorithms use heuristic information, but A* is more suitable for emergency medicine delivery because it considers both actual cost and estimated distance. This results in safer, more efficient, and reliable route planning in dynamic environments.

2. Smart City Traffic Signal Optimization

Introduction

A smart city uses AI to optimize traffic signals at many intersections. The goal is to reduce waiting time, fuel consumption, and traffic congestion. Since traffic changes continuously and the search space is very large, optimization algorithms are more suitable than traditional search methods.

Formulation as an Optimization Problem

The objective is to find the best traffic signal timings that minimize:

- Total waiting time.
- Fuel consumption.
- Traffic congestion.

Traditional search is inefficient because the number of possible signal combinations is extremely large and traffic conditions change frequently.

i. Hill Climbing Algorithm

Hill Climbing starts with an initial traffic signal plan and continuously makes small changes to improve traffic flow. It accepts only changes that produce a better solution.

Limitations

- May stop at a local optimum.
- Cannot explore worse solutions that may lead to better results.
- Performance depends on the starting solution.

Analysis: Hill Climbing is simple and fast but may not find the best overall traffic signal plan.

ii. Local Maxima, Plateaus and Ridges

- **Local Maxima:** The algorithm finds a good solution but misses a better one. For example, reducing congestion in one area while causing congestion elsewhere.
- **Plateaus:** Many signal settings produce similar results, making it difficult to decide the next move.
- **Ridges:** Better solutions require multiple coordinated changes, which Hill Climbing cannot easily perform.

Analysis: These problems can prevent the algorithm from reaching the global optimum.

iii. Simulated Annealing and Genetic Algorithms

Simulated Annealing

- Occasionally accepts worse solutions.
- Escapes local maxima.
- Increases the chance of finding a better overall solution.

Genetic Algorithm

- Maintains multiple signal plans at the same time.
- Uses selection, crossover, and mutation to generate improved solutions.

- Adapts well to large and complex traffic networks.

Analysis: Both methods overcome the limitations of Hill Climbing and search more effectively.

iv. Recommended Approach

Recommended: Genetic Algorithm

Justification

- Handles large search spaces efficiently.
- Optimizes multiple objectives simultaneously.
- Produces high-quality solutions.
- Adapts well to changing traffic conditions.
- Suitable for real-time smart city applications.

Conclusion

Traffic signal optimization is a complex optimization problem where traditional search methods are not practical. Although Hill Climbing is simple, it can become trapped in local optima. Genetic Algorithms provide better scalability, adaptability, and overall performance, making them the most suitable approach for smart city traffic management.

3. Autonomous Mars Rover – Online Search Agent

Introduction

An autonomous Mars rover explores unknown terrain to collect samples while avoiding hazards such as rocks and craters. Since it has limited sensing capability and no complete map, it must make decisions as it gathers new information.

i. Why Online Search is Required

Online search is suitable because the rover does not know the complete environment before starting.

- Explores and searches at the same time.
- Makes decisions using current observations.
- Updates its path when new obstacles are detected.

Analysis: Offline search is not suitable because it requires a complete map before planning.

ii. Challenges of Partial Observability and Dynamic Environment

- The rover can only sense nearby areas.
- Hidden obstacles may appear during movement.
- The environment is uncertain and constantly changing.
- Wrong decisions may increase travel time or risk.

Analysis: These challenges require the rover to adapt continuously while ensuring safe navigation.

iii. Building and Updating the Internal Model

The rover maintains an internal map of the environment.

- Stores information about explored areas.
- Updates obstacle locations as new data is received.
- Uses the updated model to choose safer routes.

Analysis: The internal model helps the rover improve future decisions and avoid repeated mistakes.

iv. Online Search vs. Uninformed and Informed Search

- **Online Search:** Learns while exploring and works without a complete map.

- **Uninformed Search:** Does not use heuristic information and is less efficient.
- **Informed Search:** Uses heuristics but generally requires knowledge of the environment.

Analysis: Online search is more suitable for unknown environments than traditional search methods.

v. Recommended Approach

Recommended: Online Search Agent

Justification

- Works effectively with incomplete information.
- Adapts to newly discovered obstacles.
- Supports real-time decision making.
- Improves safety and navigation efficiency.

Conclusion

An autonomous Mars rover operates in an uncertain environment where complete information is unavailable. Online search allows the rover to learn, adapt, and update its decisions while exploring. Therefore, it is the most suitable approach for ensuring safe and efficient navigation.

4. University Exam Timetable Generation using Constraint Satisfaction Problem (CSP)

Introduction

A university needs to generate an exam timetable while satisfying several constraints such as avoiding exam clashes, managing limited halls, faculty availability, and special student requirements. This can be modeled as a Constraint Satisfaction Problem (CSP).

Formulation as a CSP

- **Variables:** Exam subjects to be scheduled.
- **Domains:** Available dates, time slots, and exam halls.
- **Constraints:** No overlapping exams, limited halls, faculty availability, subject order, and special accommodations.

Analysis: A valid timetable is obtained only when all constraints are satisfied.

i. Variables, Domains and Constraints

Variables

- Each course or exam.

Domains

- Available time slots and exam halls.

Constraints

- Students should not have overlapping exams.
- Hall capacity should not be exceeded.
- Faculty must be available.
- Some exams must be conducted before others.

Analysis: These constraints ensure a feasible and conflict-free timetable.

ii. Backtracking Search

Backtracking assigns a time slot to one exam at a time.

- Checks constraints after each assignment.

- If a conflict occurs, it returns to the previous step.
- Tries another possible assignment until a valid timetable is found.

Analysis: It guarantees a solution if one exists but may take longer for large problems.

iii. Constraint Propagation (Forward Checking)

Forward checking removes invalid choices after each assignment.

- Detects conflicts early.
- Reduces unnecessary searching.
- Improves the efficiency of backtracking.

Analysis: It speeds up timetable generation by preventing invalid assignments.

iv. Real-World Challenges and Best Strategy

Challenges

- Large number of courses and students.
- Frequent timetable changes.
- Multiple constraints to satisfy simultaneously.

Recommended Strategy

Backtracking with Forward Checking

Justification

- Reduces the search space.
- Handles multiple constraints efficiently.
- Produces valid timetables for large universities.

Conclusion

Exam timetable generation is a complex CSP involving many variables and constraints. Backtracking combined with forward checking provides an efficient and reliable solution by reducing conflicts and improving scalability, making it suitable for real-world university scheduling.

5. Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning)

Introduction

A game AI must make intelligent decisions against human players within a limited time. Since many possible moves are available, the AI uses Minimax and Alpha-Beta Pruning to choose the best move efficiently.

i. Minimax Algorithm

Minimax models the game as a search tree with two players:

- **MAX** tries to maximize the score.
- **MIN** tries to minimize the score.
- The AI evaluates possible moves and selects the one with the best outcome.

Analysis: Minimax helps the AI make optimal decisions by assuming the opponent also plays optimally.

ii. Computational Challenges

- The game tree grows rapidly as possible moves increase.
- Searching every node requires more time and memory.
- Real-time games need faster decision making.

Analysis: Large game trees make pure Minimax computationally expensive.

iii. Alpha-Beta Pruning

Alpha-Beta Pruning improves Minimax by eliminating branches that cannot affect the final decision.

- Reduces the number of nodes evaluated.
- Produces the same optimal result as Minimax.
- Increases search speed without reducing accuracy.

Analysis: By pruning unnecessary branches, the AI can make faster decisions within time limits.

iv. Evaluation Function

The evaluation function assigns a score to each game state based on important factors such as:

- Number of available resources.
- Strength of units.
- Territory controlled.
- Distance from the objective.
- Health or survival of important units.

Justification: These factors help the AI compare different game states and choose the strongest move.

v. Recommended Strategy

Recommended: Minimax with Alpha-Beta Pruning

Justification

- Produces high-quality decisions.
- Reduces computation time.
- Suitable for large game trees.
- Balances optimality and real-time performance.

Conclusion

Minimax provides intelligent decision-making by considering the best possible moves, while Alpha-Beta Pruning significantly reduces unnecessary computations. Together, they enable game AI to make fast, accurate, and efficient decisions, making them the most suitable approach for real-time strategy games.